Report of I2P II Final Project.

- To start off, the main project is actually built on the Negamax algorithm. The Negamax algorithm is basically Minimax, but our opportunities are the opponent's losses, so the points in which we can get will be the points the opponent lost. In the code, there will be something like "score = same ? raw_score : -raw_score; " which at every recursion, it will maximize or minimize the points that our AI can get or the opponent can get/
- Next up, based on the Negamax algorithm, I implemented Alpha - Beta Pruning. But not the same as the theories, where Alpha least amount of points that we can get, Beta is the largest amount of points that the opponents can give us, and the algorithm will trim off the trees where the leaves surpass the criteria in which Alpha <= N <= Beta. I implemented the Nega AlphaBeta instead. The logic is basically the same, but instead of just implementing one side, the Nega AlphaBeta will be based on the turns change to help prune the tree. So our top amount, which is Beta will be the least amount that the opponent will try to get (which the amount will be -Beta) and our least amount will be their greatest amount (we are guaranteed to get these amount so they cannot surpass -Alpha) so every recursion, we will swap the values of Alpha and Beta for each participant, after that we will update the alpha value if the current value is higher than the original alpha, and if alpha is >= beta, that branch will be cut off. (We can understand this by imagining that if a move gives so many points to us, the opponents won't let it happen, and then the moves later in that branch won't be evaluated further)
- After that, we have PVS (Principal Variation Search). Normally, the PVS will assume that the leftmost branch (which, presumably, is the first move) is the best move that can happen. It will AlphaBeta through that node and find the exact value of it, next step it will go through other moves and have Null Windows which force the beta values to only 1 point higher than alpha, and it will test al the moves. If that move's score is <= 50, it will be cut off; otherwise, the PVS will go to that move and run AlphaBeta from there to find the exact score and see if that move is really good. And the Nega PVS version will also have the same exact idea as the original version, but instead of only going from [alpha, alpha+1] to [alpha, alpha+1], it will have the same recursion logic as the Nega AlphaBeta, which swaps alpha and beta based on the players. And in the code will be something like [-alpha - 1, alpha] and looping recursively.
- After implementing PVS, my AI is still losing against Minimax-weak, Minimax-strong and the visible boss, so I have also implemented Quiescence search. Quiescence search is based on Nega AlphaBeta algorithm. After PVS goes to the deepest depth, it will go through the horizon effect at such a time when the depth is <= 0, the AI cannot know if there are any of our pieces are gonna be caught, so the points at such depth will still be high and after certain moves, it will turn out to be that move was so bad. So the Quiescence search is implemented so at every depth <=0, it will get activated. First, it will get a permission to not capture the opponent's piece, it will take the score of the present, comparing with the upper bound Beta, if it surpasses the Beta, that branch will be cut of immediately. Then, if the present score isnt surpassing the upper bound it will go deeper to that move, the main part is it will only find the capture moves and return the points after all the captures is evaluated until no more capture moves can be found, it will return the score of that moment.

- After implementing Quiescence into NegeMax PVS, I found out that the PVS only focuses on how fast the pruning part, so another problem happened. Since the PVS will assume that the first move is the best move, so what would happen if that move is actually really bad? It would take a lot more time to expand and evaluate other moves to find out which might pass the limit of 2000ms per move. So I ask several questions to ChatGPT and some other AI, and at last, I have the Move Ordering Framework implemented inside. The Move Ordering Framework actually helps sort the moves from best to worst, and the sorting procedure includes 5 priorities and the moves before being searched will undergo these 5 conditions.

"The first optimization is looking up the **Transposition Table (TT)**. The TT is a hash map created to store **previously evaluated positions** during the PVS search, caching four key pieces of information: depth, score, best_move, and flag.

The table is accessed at the very beginning of the `eval_ctx` function and updated at its end. First, the AI checks if the current position exists in the TT. If a match is found and the cached depth is greater than or equal to the current search depth, it inspects the cached flag:

- **`TT_EXACT`**: The cached score is an exact minimax value (it fell strictly inside the Alpha-Beta bounds). The AI will return this score immediately without further searching.
- **`TT_LOWER`**: The cached score is a lower bound (caused by a previous Beta cut-off). If this score is greater than or equal to the current Beta, the branch is pruned. Otherwise, it can be used to raise the lower bound (Alpha).
- **`TT_UPPER`**: The cached score is an upper bound (meaning all moves were previously found to be poor). If this score is less than or equal to the current Alpha, the branch is pruned. Otherwise, it can be used to lower the upper bound (Beta).

Even when the depth requirement is not met, the TT remains highly valuable. It provides the `best_move` found at a shallower depth, which is then prioritized in Move Ordering to drastically improve Alpha-Beta pruning efficiency at the current deeper search. Finally, at the end of `eval_ctx`, the calculated results of the current node are stored back into the TT for future lookups."

- "The second priority within the Move Ordering Framework is the **Immediate Win heuristic**. The logic for this algorithm is straightforward: it checks if any available move can directly capture the opponent's King. If such a fatal move exists, the engine immediately assigns it a massive score, specifically `P_MAX - ply`. Subtracting `ply` (the current depth from the root) is a crucial detail; it teaches the AI to prefer the fastest possible win. Because this returned score is phenomenally large, it instantly exceeds the Beta bound, triggering a strict **Beta cut-off**. This completely terminates the evaluation of the current node, reducing a massive amount of unnecessary recursive loops. Furthermore, this Immediate Win check does not only act as a priority in move ordering. It also serves as a powerful short-circuit condition inside the `search`, `eval_ctx`, and `quiescence` functions. By verifying this condition early,

the engine can instantly prune the search tree the moment a winning move is spotted, significantly saving computation time."

- The next optimization in the Move Ordering framework is **MVV-LVA (Most Valuable Victim - Least Valuable Aggressor)**. As the name suggests, this heuristic determines which capture moves should be evaluated first. The score for each capture is calculated using the formula: `10 * piece_value(victim) - piece_value(aggressor)`. This ensures that the engine strongly prioritizes capturing a high-value piece with a low-value piece. By evaluating these highly profitable captures first, the search quickly discovers moves with exceptionally high scores. For example, if a pawn can capture a queen, the resulting high score will instantly trigger a Beta cut-off, significantly speeding up the pruning process. Furthermore, MVV-LVA effectively helps the PVS algorithm sort the move list. For instance, given the choice between a pawn capturing a rook and a bishop capturing a pawn, this logic guarantees that the 'pawn takes rook' scenario is prioritized and searched first."
- "The fourth optimization is the **Killer Move heuristic**. Whenever a Beta cut-off is triggered by a non-capture move (a quiet move), the AI records it as a "killer move" for that specific depth, or "ply". To maximize efficiency, the system maintains exactly two killer move slots per ply. When a new killer move is discovered, it undergoes a shifting update mechanism: first, it checks to ensure the move is not already recorded to avoid duplicates. If it is a completely new killer move, the move currently in the primary slot is shifted down to the secondary slot (discarding the older one), and the newly found move takes over the primary slot. Later, when the AI evaluates other branches at that exact same ply, it checks the generated legal moves against these two stored slots. If a match is found, this move is bumped up to a very high priority in the Move Ordering list. By evaluating these proven moves first, the PVS algorithm has a high probability of causing another quick Beta cut-off, saving significant computation time."
- "The final optimization is the **History Heuristic**, which evaluates and scores all quiet (non-capture) moves on a global scale. Whenever a quiet move successfully causes a Beta cut-off, it is recorded in a history table and awarded a bonus score. This bonus is scaled based on the search depth; the greater the depth at which the cut-off occurs, the higher the score it receives. During the move ordering process, this heuristic acts as a critical fallback. For any moves that do not fall into the higher-priority categories—such as Transposition Table matches, immediate wins, killer moves, or valuable captures—the AI relies on this global history. It sorts these remaining quiet moves based on their accumulated history scores, naturally prioritizing the moves that have historically proven to be the most effective across the entire search tree."